PROPOSAL TUGAS 1

MATA KULIAH PENGEMBANGAN APLIKASI BERBASIS WEB

TYPING SPEED BATTLE

Aplikasi Uji Kecepatan Mengetik Berbasis JavaScript Murni dengan Bantuan AI Generatif

Disusun Oleh:

Immanuel Parasian Lumbantobing (11S24035)

Dosen Pengampu:

Arie Satia Dharma, S.T, M.Kom.

Link Repositori Github:

https://github.com/ifs24035/11S24035-pabwe2026-typing-speed-battle

PROGRAM STUDI INFORMATIKA

FAKULTAS INFORMATIKA DAN TEKNIK ELEKTRO

INSTITUT TEKNOLOGI DEL

2026

1. PENDAHULUAN

1.1 Latar Belakang

Kemampuan mengetik cepat dan akurat merupakan keterampilan dasar bagi mahasiswa, terutama di bidang teknologi informasi. Kegiatan pemrograman, penulisan dokumentasi, hingga komunikasi teknis semuanya bergantung pada keterampilan ini. Namun, banyak pengguna tidak mengetahui kecepatan dan tingkat akurasi mengetik mereka, serta tidak memiliki sarana latihan yang terukur dan menyenangkan.

Di sisi lain, perkembangan Generative Artificial Intelligence (AI) telah mengubah cara pengembang perangkat lunak merancang, menulis, menguji, dan memperbaiki kode program. Alat seperti Antigravity IDE, Google Gemini, ChatGPT, GitHub Copilot, dan Claude dapat membantu memahami konsep, menghasilkan potongan kode, menemukan kesalahan, dan memberikan rekomendasi perbaikan. Meskipun demikian, hasil AI tidak selalu benar. Pengembang tetap harus memberi instruksi yang jelas, mengevaluasi hasilnya secara kritis, memverifikasi secara teknis, dan bertanggung jawab atas keputusan akhir.

Berdasarkan hal tersebut, proposal ini mengusulkan pengembangan **Typing Speed Battle**, yaitu aplikasi web sederhana untuk mengukur kecepatan mengetik (WPM, words per minute) dan akurasi, dengan kata acak, dua mode kata (Umum dan IT), serta leaderboard lokal. Aplikasi dibangun menggunakan HTML, CSS, dan JavaScript murni (tanpa framework) dengan AI generatif sebagai asisten pemrograman. Seluruh proses dalam siklus pengembangan, mulai dari analisis, perancangan, penulisan kode, pengujian, hingga debugging, didokumentasikan dalam proposal ini.

1.2 Rumusan Masalah

1. Bagaimana merancang aplikasi uji kecepatan mengetik berbasis web dengan JavaScript murni tanpa framework?
2. Bagaimana menghitung WPM dan akurasi secara benar, konsisten, dan tahan terhadap kasus tepi?
3. Bagaimana memanfaatkan AI generatif secara efektif dan kritis dalam merancang, menulis, menguji, dan memperbaiki kode?
4. Bagaimana mengidentifikasi dan memperbaiki kesalahan (bug) pada kode yang dihasilkan AI, serta menjelaskan keputusan yang diambil?

1.3 Tujuan

1. Menghasilkan aplikasi Typing Speed Battle yang berjalan di peramban tanpa framework dan tanpa library eksternal.
2. Menerapkan konsep inti JavaScript: manipulasi DOM, penanganan event, timer, pengelolaan state, modul (ES Modules), dan localStorage.
3. Mendokumentasikan proses kerja bersama AI: pemberian instruksi (prompt) beserta alasannya, evaluasi kode, identifikasi kesalahan, perbaikan, dan penjelasan keputusan.
4. Menyimpan seluruh hasil (proposal dan kode program) pada repositori GitHub yang rapi, lengkap dengan cara menjalankan, inisialisasi, dan konfigurasi.

1.4 Manfaat

- **Bagi pengguna:** sarana latihan mengetik yang terukur, menyenangkan, dan menyimpan perkembangan skor.
- **Bagi penulis:** pemahaman praktis JavaScript murni serta pengalaman bekerja dengan AI secara bertanggung jawab dan kritis.
- **Bagi mata kuliah:** contoh dokumentasi penggunaan AI dalam siklus pengembangan perangkat lunak yang dapat diperiksa dan diulang.

1.5 Ruang Lingkup dan Batasan

1. Aplikasi berjalan sepenuhnya di sisi klien (tanpa backend dan tanpa basis data server).
2. Data leaderboard disimpan di localStorage peramban sehingga bersifat lokal pada satu perangkat dan peramban.
3. Mode permainan: Mode Umum (kata sehari-hari) dan Mode IT (istilah teknologi informasi); durasi 30 atau 60 detik.
4. Hanya menggunakan HTML5, CSS3, dan JavaScript (ES6+); tidak memakai framework (React, Vue, dan sejenisnya) maupun library eksternal.
5. Pengujian dilakukan pada peramban modern (Chrome, Edge, Firefox) di layar laptop dan ponsel.

2. ANALISIS KEBUTUHAN

2.1 Deskripsi Singkat Aplikasi

Pengguna memilih mode kata dan durasi. Aplikasi menampilkan deretan kata acak. Saat pengguna mulai mengetik, timer berjalan. Setiap kata yang selesai diketik (ditandai dengan tombol spasi) diperiksa dan diberi warna benar atau salah. Di akhir sesi ditampilkan WPM, akurasi, jumlah kata benar, dan jumlah kata salah. Pengguna dapat menyimpan skor beserta namanya ke leaderboard lokal yang menampilkan 10 skor tertinggi.

2.2 Aktor dan Skenario Penggunaan

Aplikasi hanya memiliki satu aktor, yaitu Pemain (pengguna umum). Skenario utama adalah sebagai berikut:

1. Pemain membuka aplikasi dan memilih mode (Umum atau IT) serta durasi (30 atau 60 detik).
2. Aplikasi mengacak kata dan menampilkannya; kolom input otomatis aktif.
3. Pemain mengetik kata pertama; timer mulai berjalan secara otomatis.
4. Setiap kali menekan spasi, aplikasi menilai kata tersebut (benar atau salah) dan menyorot kata berikutnya.
5. Ketika waktu habis, aplikasi menghitung dan menampilkan WPM, akurasi, serta ringkasan kata.
6. Pemain memasukkan nama dan menyimpan skor; leaderboard diperbarui. Pemain dapat mengulang sesi kapan saja.

2.3 Kebutuhan Fungsional

Kode	Kebutuhan	Prioritas
F-01	Pengguna dapat memilih mode kata (Umum / IT).	Tinggi
F-02	Pengguna dapat memilih durasi tes (30 / 60 detik).	Tinggi
F-03	Aplikasi menampilkan kata acak yang urutannya berbeda pada setiap sesi.	Tinggi
F-04	Timer mulai otomatis pada ketikan pertama dan berhenti saat waktu habis.	Tinggi
F-05	Umpan balik visual per kata (benar, salah, dan kata aktif).	Tinggi
F-06	Aplikasi menghitung WPM dan akurasi di akhir sesi.	Tinggi
F-07	Skor dapat disimpan ke leaderboard lokal beserta nama pemain.	Sedang
F-08	Leaderboard menampilkan 10 skor tertinggi, terurut dari WPM terbesar.	Sedang
F-09	Tombol Ulangi untuk memulai sesi baru kapan saja.	Sedang
F-10	Tombol hapus leaderboard.	Rendah

2.4 Kebutuhan Non Fungsional

Kode	Aspek	Deskripsi	Cara verifikasi
NF-01	Kegunaan	Antarmuka sederhana dan responsif pada lebar layar 360 px sampai 1920 px.	Uji dengan mode perangkat di DevTools
NF-02	Performa	Tampilan tidak tersendat saat mengetik cepat; pembaruan timer tiap 100 ms.	Pengamatan dan Performance DevTools
NF-03	Keandalan	Aplikasi tidak error saat `localStorage` kosong atau berisi data rusak.	Uji dengan data rusak (Bab 7)
NF-04	Keamanan	Teks dari pengguna (nama pemain) ditampilkan tanpa dieksekusi sebagai HTML.	Uji dengan nama berisi tag HTML
NF-05	Portabilitas	Berjalan di Chrome, Edge, dan Firefox terbaru melalui server statis tanpa instalasi dependensi.	Uji di tiga peramban
NF-06	Keterbacaan kode	Kode dibagi per tanggung jawab, bernama jelas, dan diberi komentar seperlunya.	Tinjauan kode (code review)

2.5 Rumus Perhitungan

Perhitungan mengikuti konvensi umum pada aplikasi uji mengetik, yaitu satu kata standar = 5 karakter. Dengan konvensi ini, kata panjang dan kata pendek diperlakukan adil.

```
WPM       = (karakter benar / 5) / durasi dalam menit

Akurasi   = (karakter benar / total karakter diketik) x 100%
```

Contoh perhitungan. Pemain mengetik 50 karakter benar dalam 30 detik: WPM = (50 / 5) / 0,5 = **20 WPM.** Jika dari 50 karakter yang diketik terdapat 45 karakter benar, akurasi = 45 / 50 x 100% = **90%.**

2.6 Analisis Risiko

Risiko	Dampak	Mitigasi
Kode dari AI mengandung kesalahan logika atau bug tersembunyi	Hasil WPM salah, aplikasi error	Setiap fungsi diuji dengan hitungan manual dan uji otomatis sederhana
AI menyarankan library atau pendekatan di luar ketentuan tugas	Melanggar syarat tanpa framework	Batasan teknis ditulis eksplisit pada prompt; kode diperiksa sebelum dipakai
Penulis menyalin kode tanpa memahami	Sulit menjelaskan dan memperbaiki kode	Setiap blok kode dijelaskan ulang dengan kata sendiri dan dicatat keputusannya
Data leaderboard rusak atau hilang	Aplikasi berhenti bekerja	Penanganan `try/catch` dan nilai bawaan daftar kosong
Perbedaan perilaku antar peramban	Tampilan atau fungsi tidak konsisten	Pengujian lintas peramban dan penggunaan fitur standar saja

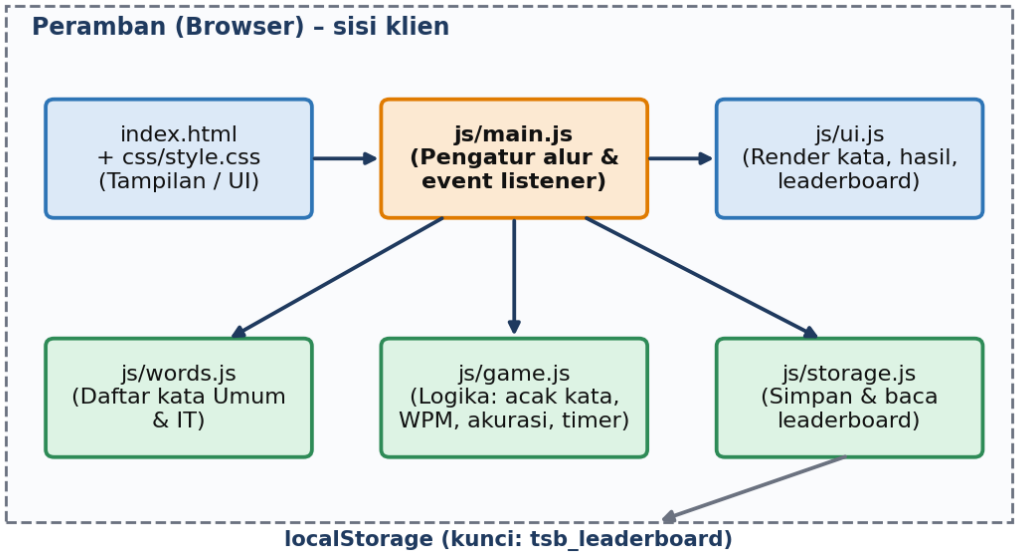
3. PERANCANGAN

3.1 Teknologi yang Digunakan

Teknologi / Alat	Fungsi
HTML5	Struktur halaman dan elemen antarmuka
CSS3 (Flexbox / Grid)	Tata letak responsif dan gaya visual
JavaScript ES6+ (ES Modules)	Logika permainan, manipulasi DOM, event, dan timer
`localStorage` (Web Storage API)	Menyimpan leaderboard di peramban
Git dan GitHub	Pengendalian versi dan penyimpanan proposal serta kode
Visual Studio Code + Live Server	Penyuntingan kode dan server lokal
Browser DevTools	Debugging, inspeksi DOM, dan uji responsif
AI generatif (utama: Antigravity IDE, Gemini dan Claude; opsional: ChatGPT, GitHub Copilot)	Asisten merancang, menulis, menguji, dan memperbaiki kode

3.2 Arsitektur Aplikasi

Aplikasi dirancang modular dengan prinsip satu modul satu tanggung jawab. Modul main.js bertindak sebagai pengatur alur yang memanggil modul lain, sedangkan logika perhitungan dipisahkan dari tampilan agar mudah diuji.



Modul	Tanggung jawab
index.html, style.css	Kerangka halaman dan tampilan; tidak berisi logika.
main.js	Pengatur alur: menangkap event, memperbarui state, memanggil modul lain.
game.js	Fungsi murni: acak kata, hitung WPM dan akurasi, nilai kata, serta timer.
words.js	Daftar kata untuk Mode Umum dan Mode IT.

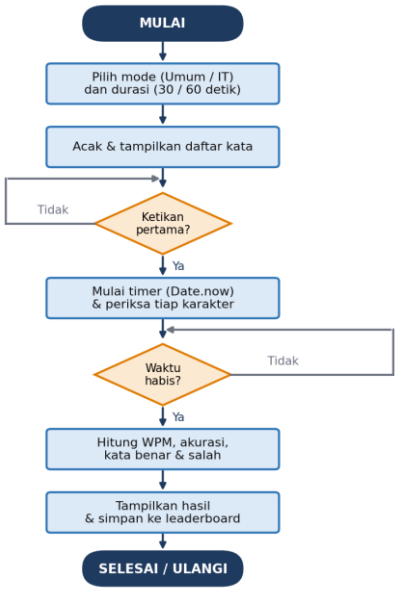
Modul	Tanggung jawab
storage.js	Membaca, menyimpan, dan menghapus leaderboard di `localStorage`.
ui.js	Menampilkan kata, hasil, dan leaderboard ke DOM.
config.js	Pengaturan terpusat (durasi, jumlah kata, batas skor, kunci penyimpanan).

3.3 Struktur Folder

```
typing-speed-battle/
├── index.html            # struktur halaman
├── css/
│   └── style.css         # tampilan & responsif
├── js/
│   ├── config.js         # pengaturan aplikasi
│   ├── words.js          # daftar kata (Umum & IT)
│   ├── game.js           # logika: acak, WPM, akurasi, timer
│   ├── storage.js        # leaderboard di localStorage
│   ├── ui.js             # render ke DOM
│   └── main.js           # alur & event listener
├── tests/
│   └── game.test.js      # uji fungsi murni (Node.js)
├── docs/
│   ├── proposal.docx     # dokumen proposal
│   └── images/           # tangkapan layar prompt & hasil
├── package.json          # hanya {"type": "module"}, tanpa dependensi
├── .gitignore
└── README.md
```

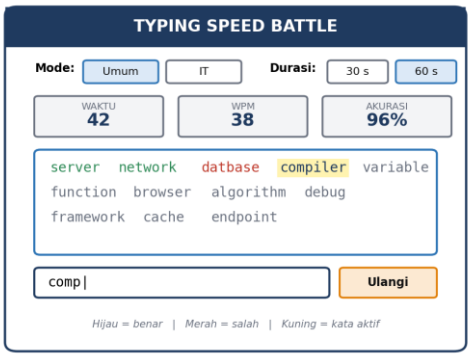
3.4 Alur Aplikasi

Alur berikut menggambarkan jalannya satu sesi permainan, mulai dari pemilihan mode hingga hasil dan leaderboard.



3.5 Rancangan Antarmuka

Rancangan antarmuka dibuat sederhana dan fokus pada area kata. Warna hijau menandai kata benar, merah kata salah, dan kuning kata yang sedang diketik. Rancangan ini menjadi acuan saat memberi instruksi pembuatan HTML dan CSS kepada AI.





3.6 Struktur Data

Leaderboard disimpan sebagai array objek pada satu kunci localStorage. Data disusun terurut menurun berdasarkan WPM, dan jika sama, berdasarkan akurasi, serta dibatasi 10 entri.

```
// Isi localStorage["tsb_leaderboard"] (array, maksimum 10 objek)
[
  { "name": "Nama A", "wpm": 58, "accuracy": 97,
    "mode": "it", "duration": 60, "date": "2026-10-20T09:15:00.000Z" },
  { "name": "Nama B", "wpm": 51, "accuracy": 94,
    "mode": "umum", "duration": 30, "date": "2026-10-20T09:20:00.000Z" }
]
```

3.7 Keputusan Desain Utama

- ES Modules dipilih agar kode terpisah per tanggung jawab. Konsekuensinya, aplikasi dijalankan melalui server statis lokal (bukan membuka berkas langsung), dan hal ini didokumentasikan di README.
- Timer berbasis [[Date.now()]], bukan sekadar pengurang angka, agar waktu tetap akurat walaupun peramban menunda interval.
- Timer dimulai pada ketikan pertama, bukan saat halaman dimuat, supaya pengguna punya waktu bersiap.
- Kata yang belum selesai diketik saat waktu habis tidak dihitung, agar penilaian konsisten dan sederhana.
- Akurasi dihitung per karakter, termasuk karakter spasi, sehingga kesalahan ketik terlihat pada persentase.

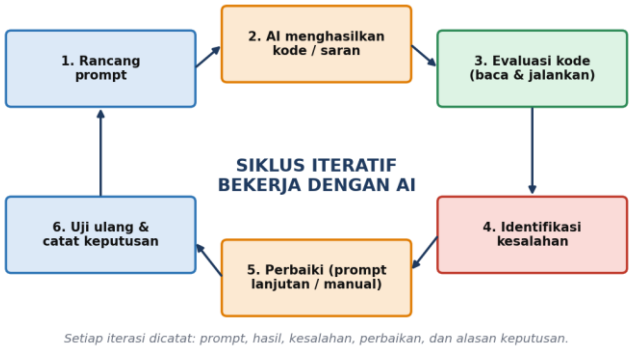
4. STRATEGI PEMANFAATN AI GENERATIF

4.1 Peran AI dan Tanggung Jawab Pengembang

AI diposisikan sebagai asisten, bukan pengganti pengembang. AI membantu mengusulkan rancangan, menulis draf kode, menjelaskan konsep, dan menganalisis bug. Sementara itu, pengembang tetap bertanggung jawab untuk (1) menetapkan kebutuhan dan batasan, (2) membaca dan memahami setiap kode sebelum dipakai, (3) menjalankan dan menguji hasilnya, dan (4) mengambil keputusan akhir serta menjelaskannya.

4.2 Siklus Kerja Bersama AI

Setiap fitur dikerjakan melalui siklus iteratif enam langkah berikut. Siklus ini menjamin bahwa tidak ada kode yang diterima tanpa evaluasi.



Setiap iterasi dicatat: prompt, hasil, kesalahan, perbaikan, dan alasan keputusan.

4.3 Prinsip Penyusunan Prompt

Prinsip	Penjelasan	Contoh pada proyek ini
Peran dan konteks	AI menyesuaikan jawaban dengan situasi	“Saya mahasiswa membuat tugas aplikasi web Typing Speed Battle.”
Batasan teknis eksplisit	Mencegah solusi di luar ketentuan	“Hanya HTML, CSS, dan JavaScript murni; tanpa library.”
Tugas dipecah kecil	Hasil lebih mudah diperiksa	“Buat game.js saja dulu, tanpa DOM.”
Format keluaran jelas	Hasil rapi dan mudah dibandingkan	“Satu file per blok kode; daftar bernomor.”
Minta alasan, bukan hanya kode	Menumbuhkan pemahaman dan bahan penjelasan keputusan	“Jelaskan mengapa memilih pendekatan ini.”
Sertakan bukti saat debugging	Diagnosis lebih tepat	Kode terkait, gejala, dan perilaku yang diharapkan
Minta cara verifikasi	Hasil tidak diterima begitu saja	“Beri contoh uji untuk membuktikan perbaikan.”

4.4 Perbandingan Prompt Kurang Baik dan Prompt Baik

Aspek	Prompt kurang baik	Prompt baik
Isi prompt	“Buatkan aplikasi typing test pakai JavaScript.”	Menyebut konteks tugas, batasan (tanpa framework), fitur, struktur modul, dan format keluaran.
Kemungkinan hasil	Satu berkas panjang, bisa memakai library, tanpa penanganan error, sulit dipahami.	Kode modular, sesuai batasan, ada penanganan kasus tepi, dan disertai alasan.
Kemudahan evaluasi	Sulit: terlalu banyak yang harus diperiksa sekaligus.	Mudah: diperiksa per modul dengan kriteria yang jelas.

4.5 Kriteria Evaluasi Kode Hasil AI

Kriteria	Pertanyaan yang diajukan	Cara memeriksa
Kebenaran fungsional	Apakah hasil sesuai rumus dan spesifikasi?	Hitung manual 3–5 contoh dan bandingkan dengan keluaran program.
Kepatuhan batasan	Adakah framework atau library terlarang?	Periksa tag `script` dan pernyataan `import`.
Kasus tepi	Bagaimana jika input kosong, waktu 0, atau data rusak?	Uji nilai batas dan data tidak valid.
Keamanan dasar	Apakah input pengguna ditampilkan dengan aman?	Cari penggunaan `innerHTML`; uji dengan nama berisi tag HTML.
Keterbacaan	Apakah nama jelas dan fungsi cukup kecil?	Tinjauan kode; rapikan bila perlu.
Pemahaman	Dapatkah saya menjelaskan setiap baris?	Jelaskan ulang dengan kata sendiri; tanyakan bagian yang belum dipahami.

4.6 Etika dan Transparansi

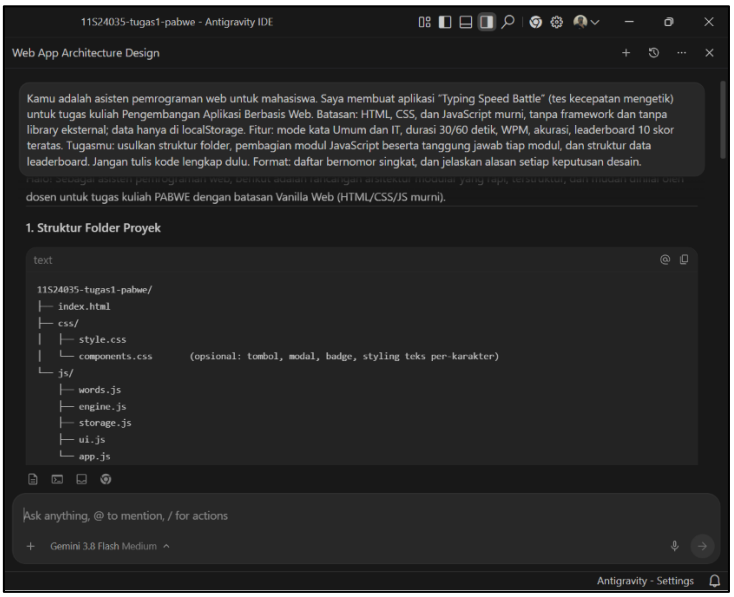
- Penggunaan AI dinyatakan secara terbuka dan seluruh prompt penting didokumentasikan.
- Kode yang dihasilkan AI hanya dipakai setelah dibaca, dipahami, dan diuji.
- Tidak mengunggah data pribadi atau informasi sensitif ke alat AI.
- Hasil akhir sepenuhnya menjadi tanggung jawab penulis, termasuk bila terdapat kesalahan.

5. DOKUMENTASI PROMPT, EVALUASI, DAN KEPUTUSAN

Bab ini memuat lima iterasi kerja bersama AI beserta prompt yang akan digunakan, alasan penyusunannya, hal yang dievaluasi, potensi kesalahan, dan keputusan yang diambil. Pada setiap iterasi disediakan kotak untuk menempelkan tangkapan layar percakapan yang sebenarnya sebagai bukti.

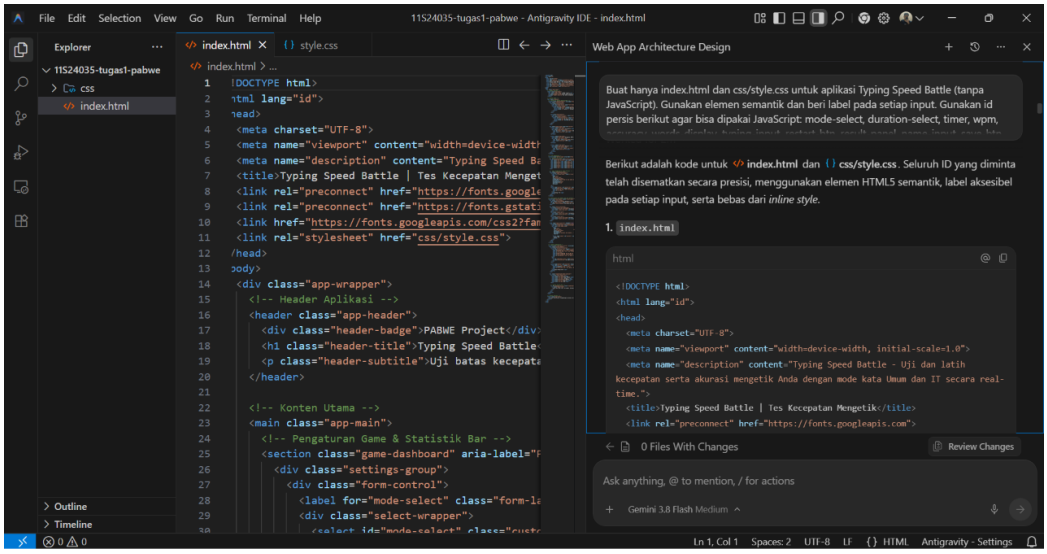
5.1 Iterasi 1: Perancangan Arsitektur

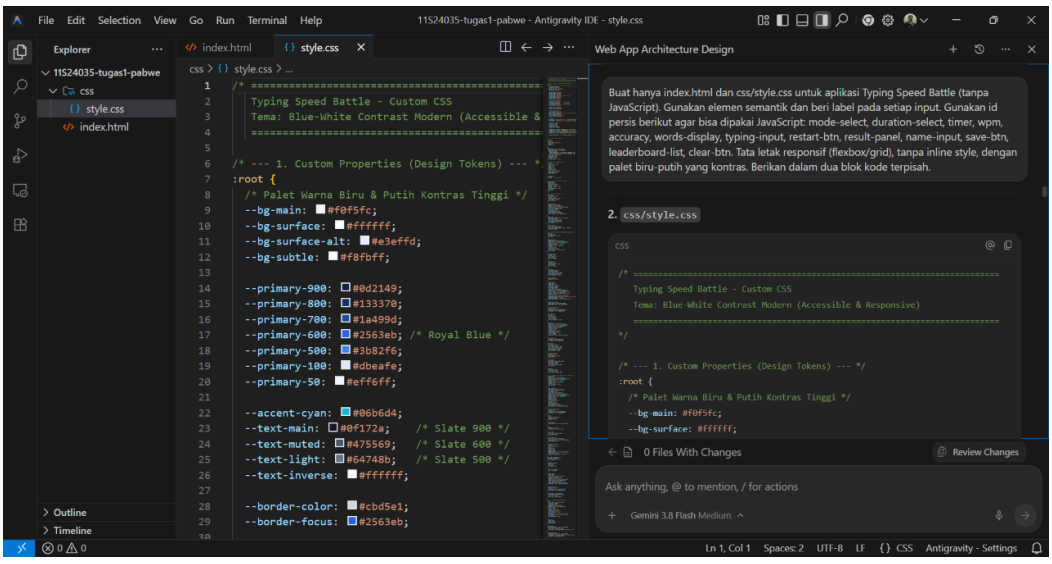
Tujuan	Menentukan struktur folder, pembagian modul, dan struktur data sebelum menulis kode.
Prompt	*Kamu adalah asisten pemrograman web untuk mahasiswa. Saya membuat aplikasi "Typing Speed Battle" (tes kecepatan mengetik) untuk tugas kuliah Pengembangan Aplikasi Berbasis Web. Batasan: HTML, CSS, dan JavaScript murni, tanpa framework dan tanpa library eksternal; data hanya di localStorage. Fitur: mode kata Umum dan IT, durasi 30/60 detik, WPM, akurasi, leaderboard 10 skor teratas. Tugasmu: usulkan struktur folder, pembagian modul JavaScript beserta tanggung jawab tiap modul, dan struktur data leaderboard. Jangan tulis kode lengkap dulu. Format: daftar bernomor singkat, dan jelaskan alasan setiap keputusan desain.*
Alasan menyusun prompt	Peran, konteks, dan batasan ditulis di awal agar AI tidak menyimpang. Larangan menulis kode lengkap membuat fokus pada rancangan yang mudah dinilai. Permintaan alasan memberi bahan untuk menjelaskan keputusan.
Hal yang dievaluasi	Apakah tiap modul memiliki satu tanggung jawab? Apakah solusi tetap sesuai batasan? Apakah struktur data cukup untuk fitur leaderboard?
Potensi kesalahan	AI menambahkan library (misalnya framework CSS atau pustaka grafik) atau memakai ES Modules tanpa menyebut bahwa halaman harus dijalankan lewat server.
Keputusan	Struktur enam modul diterima; library eksternal ditolak; ES Modules dipertahankan dengan konsekuensi dijalankan melalui server lokal dan dicatat di README.



5.2 Iterasi 2: Kerangka HTML dan CSS

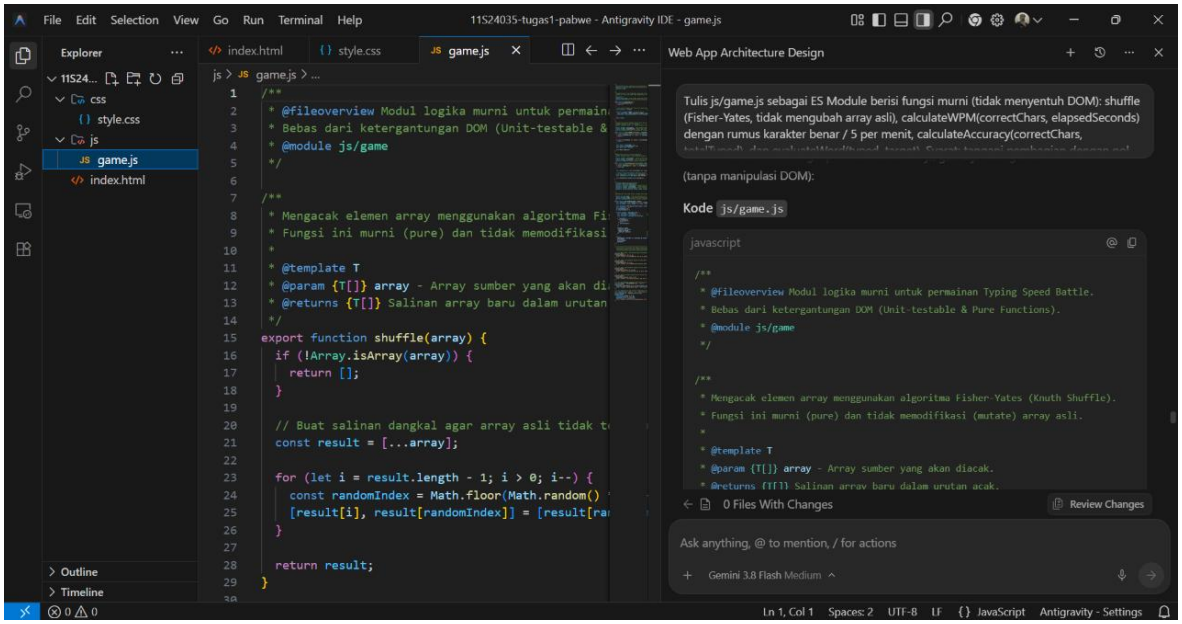
Tujuan	Membuat antarmuka statis sesuai wireframe pada Bab 3.
Prompt	*Buat hanya index.html dan css/style.css untuk aplikasi Typing Speed Battle (tanpa JavaScript). Gunakan elemen semantik dan beri label pada setiap input. Gunakan id persis berikut agar bisa dipakai JavaScript: mode-select, duration-select, timer, wpm, accuracy, words-display, typing-input, restart-btn, result-panel, name-input, save-btn, leaderboard-list, clear-btn. Tata letak responsif (flexbox/grid), tanpa inline style, dengan palet biru-putih yang kontras. Berikan dalam dua blok kode terpisah.*
Alasan menyusun prompt	Daftar id ditetapkan di awal supaya tidak terjadi ketidakcocokan nama antara HTML dan JavaScript. Memisahkan HTML-CSS dari JavaScript mempermudah evaluasi. Label dan kontras warna mendukung aksesibilitas.
Hal yang dievaluasi	Semua id ada dan unik; tampilan pada lebar 360 px dan 1366 px; kontras teks; tidak ada inline style maupun tautan CDN.
Potensi kesalahan	Nama id berbeda dari yang diminta; label terlewat; ukuran tetap (piksel) sehingga tidak responsif; menyisipkan font atau framework dari CDN.
Keputusan	Menyesuaikan id secara manual bila berbeda; menolak CDN; memperbaiki CSS agar responsif sebelum lanjut.





5.3 Iterasi 3: Logika Inti (game.js)

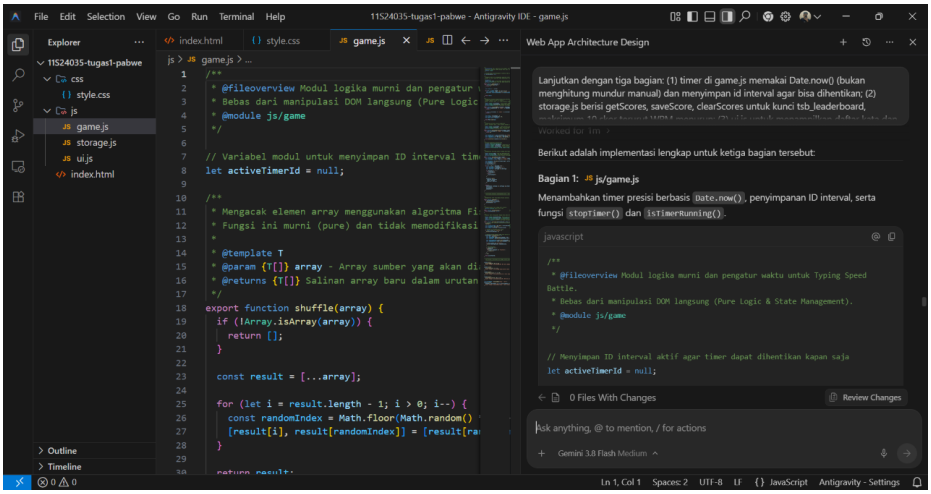
Tujuan	Membuat fungsi murni untuk mengacak kata, menghitung WPM dan akurasi, serta menilai kata.
Prompt	*Tulis js/game.js sebagai ES Module berisi fungsi murni (tidak menyentuh DOM): shuffle (Fisher-Yates, tidak mengubah array asli), calculateWPM(correctChars, elapsedSeconds) dengan rumus karakter benar / 5 per menit, calculateAccuracy(correctChars, totalTyped), dan evaluateWord(typed, target). Syarat: tangani pembagian dengan nol, beri komentar JSDoc, dan sertakan 5 pemanggilan console.assert beserta hasil yang diharapkan. Setelah kode, jelaskan kasus tepi yang kamu pertimbangkan.*
Alasan menyusun prompt	Fungsi murni mudah diuji tanpa peramban. Menyebut algoritma dan rumus tertentu mencegah AI memilih pendekatan yang keliru. Meminta kasus tepi memaksa AI membuka asumsinya.
Hal yang dievaluasi	Hitung manual: 50 karakter dalam 30 detik harus 20 WPM; array asli tidak berubah setelah `shuffle`; tidak ada nilai `NaN` atau `Infinity`.
Potensi kesalahan	Pengacakan memakai `sort(() => Math.random() - 0.5)` yang bias; pembagian dengan nol pada waktu 0; array asli ikut berubah.
Keputusan	Mengganti dengan Fisher-Yates bila AI memakai `sort` acak; menambah penjagaan waktu kurang dari atau sama dengan 0. Detail kasus ada pada Bab 8.

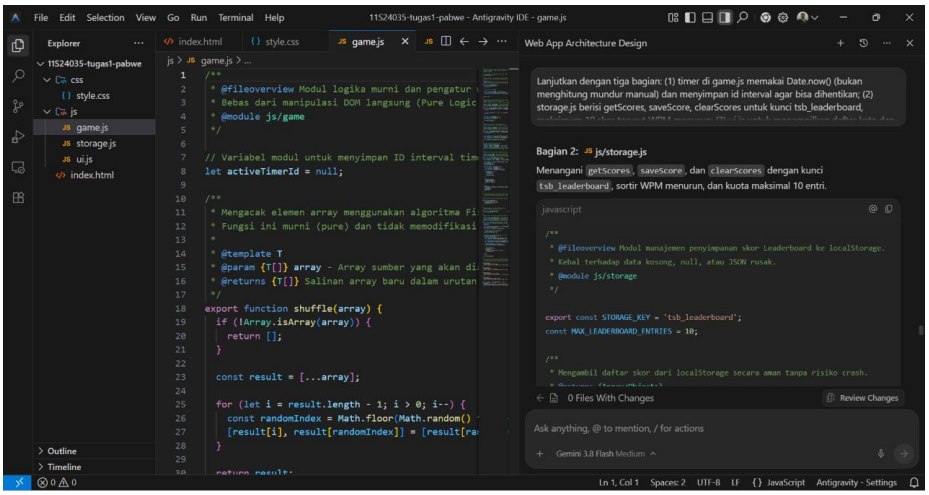


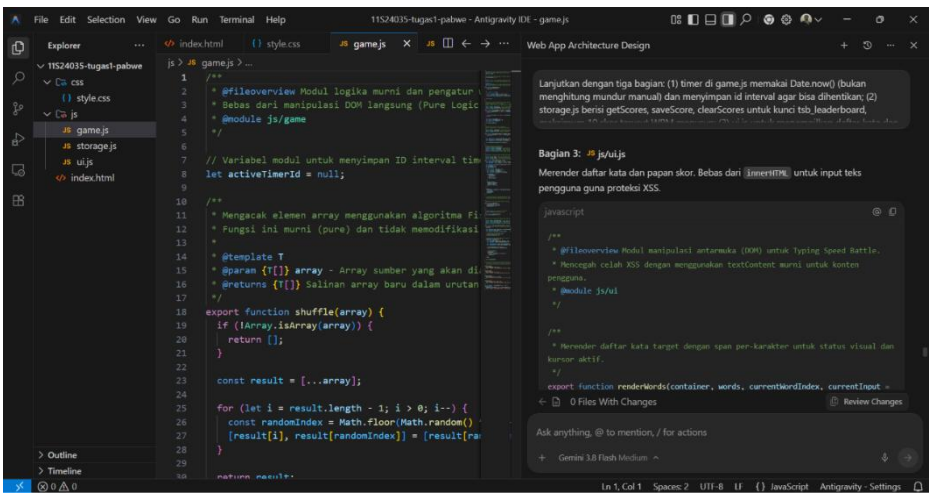
5.4 Iterasi 4: Timer, Penyimpanan, dan UI

Tujuan	Menambah timer, penyimpanan leaderboard, dan penampil antarmuka.
Prompt	*Lanjutkan dengan tiga bagian: (1) timer di game.js memakai Date.now() (bukan menghitung mundur manual) dan menyimpan id interval agar bisa dihentikan; (2) storage.js berisi getScores, saveScore, clearScores untuk kunci tsb_leaderboard, maksimum 10 skor terurut WPM menurun; (3) ui.js untuk menampilkan daftar kata dan leaderboard. Syarat: aplikasi tidak boleh crash bila localStorage kosong atau berisi JSON rusak, dan teks dari pengguna tidak boleh dimasukkan lewat innerHTML. Jelaskan alasan untuk setiap syarat.*

Alasan menyusun prompt	Syarat keandalan dan keamanan disebut eksplisit supaya tidak diabaikan AI. Permintaan alasan memungkinkan saya memeriksa apakah AI memahami konsekuensinya.
Hal yang dievaluasi	Menekan Ulangi berkali-kali sambil mengamati timer; mengisi localStorage dengan teks rusak; memasukkan nama berisi tag HTML.
Potensi kesalahan	Interval tidak dihentikan; `JSON.parse` tanpa `try/catch`; penggunaan `innerHTML` untuk teks pengguna.
Keputusan	Memperbaiki melalui prompt lanjutan atau secara manual, lalu memverifikasi dengan kasus uji pada Bab 7.

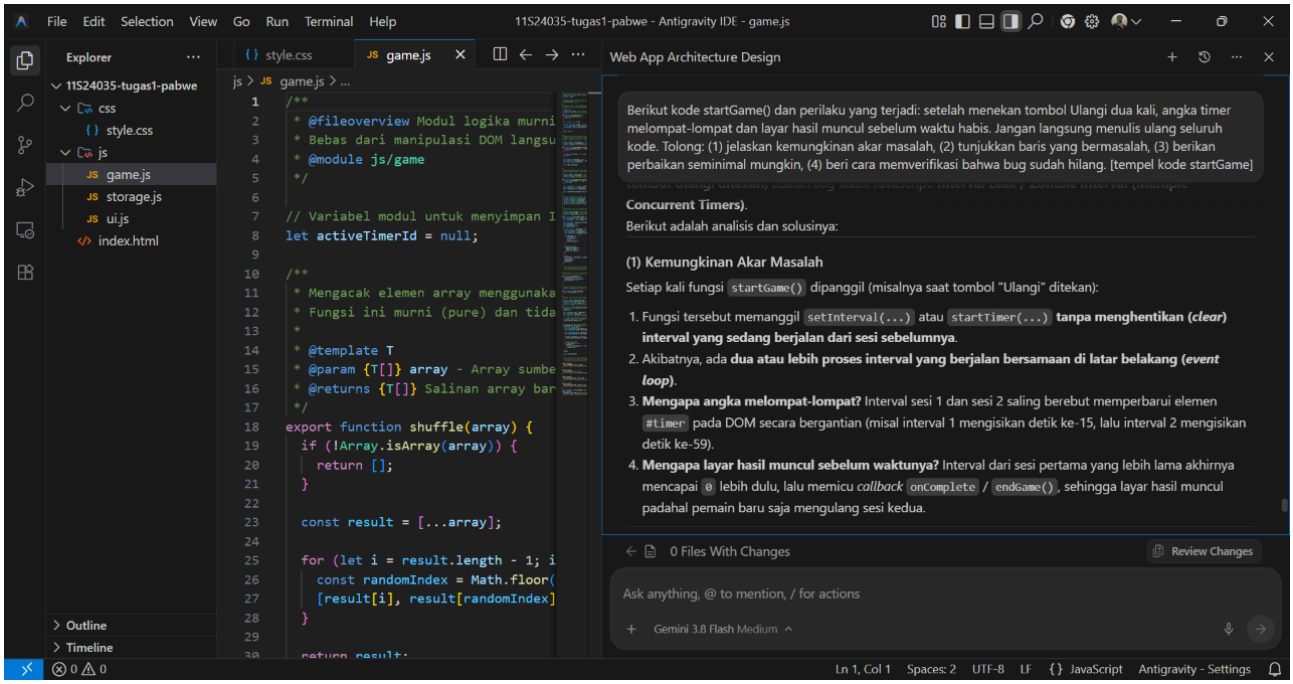






5.5 Iterasi 5: Debugging Terarah

Tujuan	Memperbaiki bug timer ganda dengan perubahan sekecil mungkin.
Prompt	*Berikut kode startGame() dan perilaku yang terjadi: setelah menekan tombol Ulangi dua kali, angka timer melompat-lompat dan layar hasil muncul sebelum waktu habis. Jangan langsung menulis ulang seluruh kode. Tolong: (1) jelaskan kemungkinan akar masalah, (2) tunjukkan baris yang bermasalah, (3) berikan perbaikan seminimal mungkin, (4) beri cara memverifikasi bahwa bug sudah hilang.*
Alasan menyusun prompt	Gejala, kode, dan harapan memberi AI bukti untuk diagnosis. Larangan menulis ulang menjaga perubahan tetap kecil dan mudah ditinjau. Permintaan verifikasi memastikan perbaikan dapat dibuktikan, bukan sekadar diklaim.
Hal yang dievaluasi	Apakah akar masalah (interval ganda) dijelaskan dengan benar? Apakah perbaikan minimal? Apakah bug dapat direproduksi sebelum dan hilang sesudah perbaikan?
Potensi kesalahan	AI menebak penyebab yang salah (misalnya event listener ganda) atau menulis ulang berkas besar sehingga bug baru muncul.
Keputusan	Hanya menerima perbaikan yang bisa direproduksi dan terbukti; penyebab dan solusinya dicatat pada Bab 8.



6. RENCANA IMPLEMENTASI KODE

Bab ini memuat rancangan kode inti yang menjadi target implementasi. Cuplikan logika (config, game, timer, dan storage) telah dijalankan dan diverifikasi di lingkungan Node.js pada saat penyusunan proposal. Cuplikan yang bergantung pada DOM (ui dan main) akan diuji di peramban pada tahap implementasi, dan seluruh kode akan disesuaikan sesuai hasil pengujian.

6.1 Konfigurasi dan Data Kata

```
// js/config.js – seluruh pengaturan aplikasi di satu tempat
export const CONFIG = {
  durations: [30, 60],          // pilihan durasi (detik)
  wordsPerRound: 120,           // jumlah kata yang ditampilkan per sesi
  maxScores: 10,                // jumlah skor di leaderboard
  maxNameLength: 20,            // batas panjang nama pemain
  storageKey: "tsb_leaderboard" // kunci localStorage
};
```

```
// js/words.js (cuplikan)
export const WORDS = {
  umum: ["rumah", "belajar", "kampus", "teman", "pagi", "buku", "cepat" /* ... */],
  it:   ["server", "network", "database", "compiler", "variable", "function",
         "browser", "algorithm", "debug", "framework", "cache", "endpoint" /* ... */]
};
```

6.2 Logika Inti

Fungsi pada game.js bersifat murni sehingga dapat diuji tanpa peramban. Fungsi evaluateWord membandingkan karakter pada posisi yang sama; sebuah kata dianggap benar hanya jika seluruh karakternya sama.

```
// js/game.js – fungsi murni (tidak menyentuh DOM)

/** Mengacak urutan array dengan algoritma Fisher-Yates. */
export function shuffle(array) {
  const arr = [...array];
  for (let i = arr.length - 1; i > 0; i--) {
    const j = Math.floor(Math.random() * (i + 1));
    [arr[i], arr[j]] = [arr[j], arr[i]];
  }
  return arr;
}

/** WPM = (karakter benar / 5) per menit. Mengembalikan 0 jika waktu <= 0. */
export function calculateWPM(correctChars, elapsedSeconds) {
  if (elapsedSeconds <= 0) return 0;
  return Math.round((correctChars / 5) / (elapsedSeconds / 60));
}

/** Akurasi (%) = karakter benar / total karakter diketik. */
export function calculateAccuracy(correctChars, totalTyped) {
  if (totalTyped === 0) return 0;
  return Math.round((correctChars / totalTyped) * 100);
}

/** Membandingkan satu kata yang diketik dengan kata target. */
export function evaluateWord(typed, target) {
  let correct = 0;
  const limit = Math.min(typed.length, target.length);
  for (let i = 0; i < limit; i++) {
```

```
    if (typed[i] === target[i]) correct++;
  }
  return { correct, total: typed.length, isCorrect: typed === target };
}
```

6.3 Timer

Timer menghitung sisa waktu dari selisih Date.now(), bukan dari pengurang angka, sehingga tetap akurat. Pemanggilan stopTimer() di awal mencegah timer ganda.

```
// js/game.js – timer berbasis waktu sistem
let timerId = null;

export function startTimer(durationSec, onTick, onFinish) {
  stopTimer();                                  // cegah timer ganda
  const startTime = Date.now();
  timerId = setInterval(() => {
    const elapsed = (Date.now() - startTime) / 1000;
    const remaining = Math.max(0, Math.ceil(durationSec - elapsed));
    onTick(remaining);
    if (remaining === 0) {
      stopTimer();
      onFinish(durationSec);
    }
  }, 100);
}

export function stopTimer() {
  clearInterval(timerId);
  timerId = null;
}
```

6.4 Penyimpanan Leaderboard

```
// js/storage.js
import { CONFIG } from "./config.js";

export function getScores() {
  try {
    const raw = localStorage.getItem(CONFIG.storageKey);
    if (!raw) return [];
    const data = JSON.parse(raw);
    return Array.isArray(data) ? data : [];
  } catch (err) {
    console.warn("Data leaderboard rusak, memakai daftar kosong.", err);
    return [];
  }
}

export function saveScore({ name, wpm, accuracy, mode, duration }) {
  const scores = getScores();
  scores.push({
    name: name.trim().slice(0, CONFIG.maxNameLength) || "Anonim",
    wpm, accuracy, mode, duration,
    date: new Date().toISOString(),
  });
  scores.sort((a, b) => b.wpm - a.wpm || b.accuracy - a.accuracy);
  localStorage.setItem(CONFIG.storageKey,
    JSON.stringify(scores.slice(0, CONFIG.maxScores)));
}

export function clearScores() {
  localStorage.removeItem(CONFIG.storageKey);
}
```

6.5 Tampilan Leaderboard

```
// js/ui.js
export function renderLeaderboard(scores, listEl) {
  listEl.replaceChildren();                        // kosongkan daftar
  scores.forEach((s, i) => {
    const li = document.createElement("li");
    li.textContent = (i + 1) + ". " + s.name + " - " + s.wpm +
                     " WPM (" + s.accuracy + "%)";   // textContent: aman dari XSS
    listEl.appendChild(li);
  });
}
```

6.6 Pengatur Alur

Timer dimulai pada ketikan pertama. Setiap spasi menandai selesainya satu kata; karakter spasi dihitung sebagai karakter benar hanya jika kata tersebut benar seluruhnya.

```
// js/main.js – pengatur alur (ringkasan)
import { shuffle, evaluateWord, calculateWPM, calculateAccuracy,
         startTimer } from "./game.js";

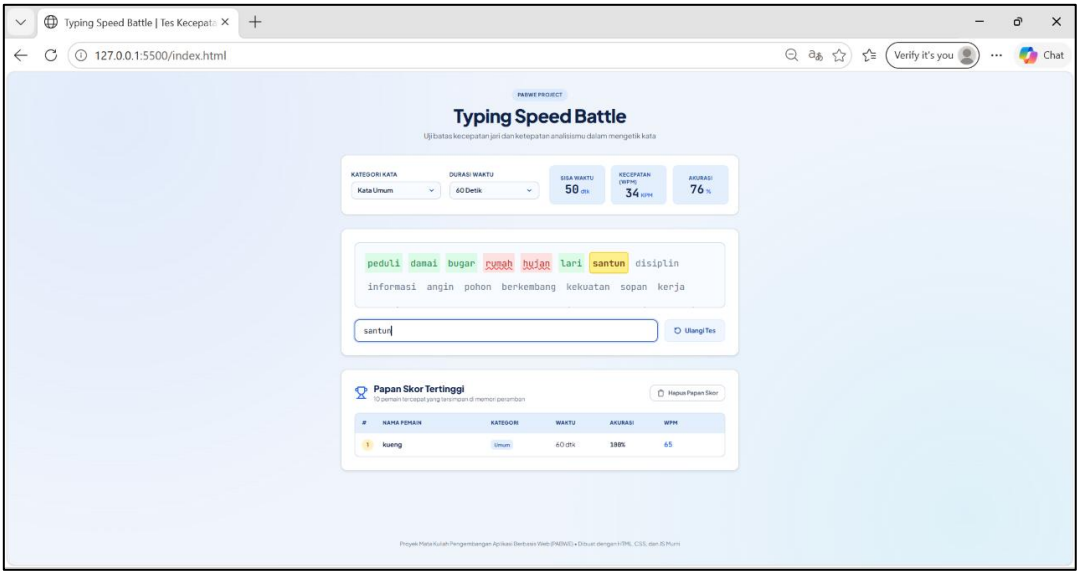
const state = { words: [], index: 0, correctChars: 0, totalTyped: 0,
                correctWords: 0, wrongWords: 0, started: false };

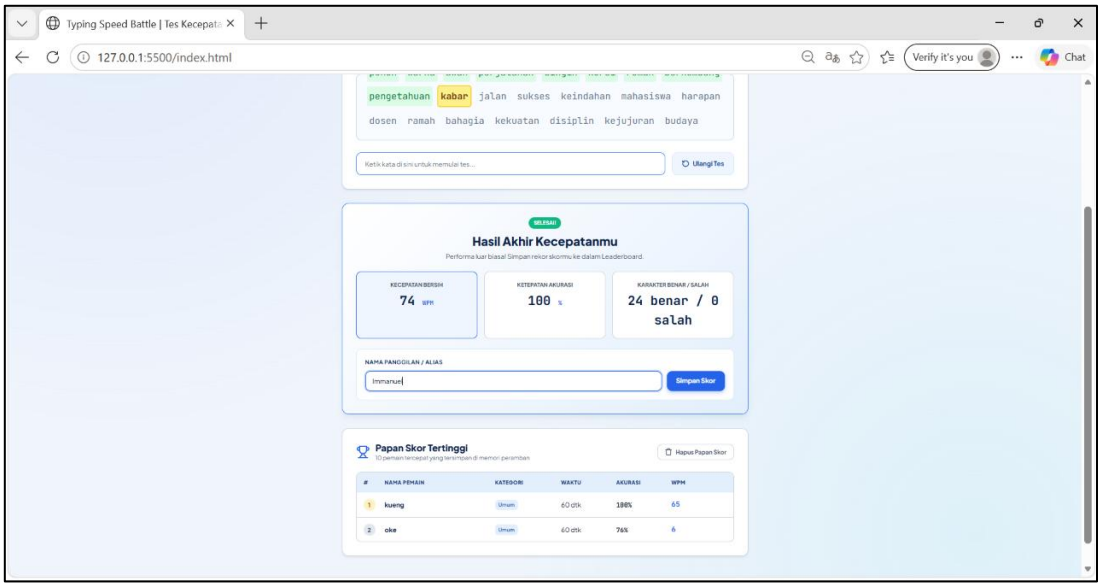
input.addEventListener("input", () => {
  if (!state.started) {                          // timer mulai saat ketikan pertama
    state.started = true;
    startTimer(duration, ui.updateTimer, finishGame);
  }
});

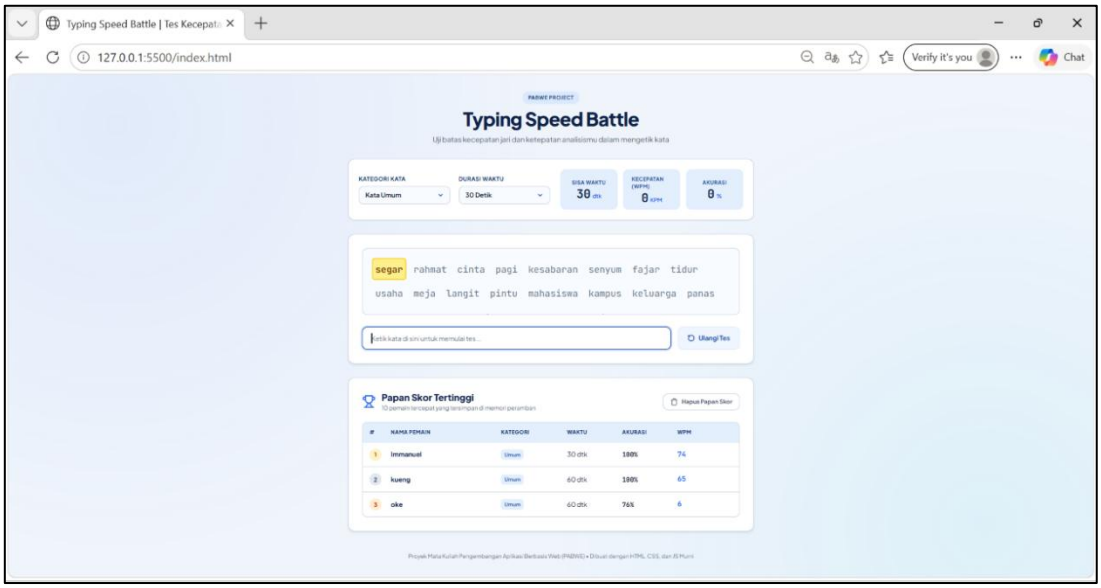
input.addEventListener("keydown", (e) => {
  if (e.key !== " ") return;
  e.preventDefault();
  const typed = input.value.trim();
  if (!typed) return;
  const r = evaluateWord(typed, state.words[state.index]);
  state.correctChars += r.correct + (r.isCorrect ? 1 : 0);   // +1 untuk spasi
  state.totalTyped   += r.total + 1;
  r.isCorrect ? state.correctWords++ : state.wrongWords++;
  ui.markWord(state.index, r.isCorrect);
  state.index++;
  input.value = "";
  ui.highlightWord(state.index);
});

function finishGame(elapsedSec) {
  input.disabled = true;
  const wpm = calculateWPM(state.correctChars, elapsedSec);
  const accuracy = calculateAccuracy(state.correctChars, state.totalTyped);
  ui.showResult({ wpm, accuracy, ...state });
}
```

Tangkapan layar aplikasi yang sudah berjalan (layar permainan dan layar hasil)







7. RENCANA PENGUJIAN

7.1 Strategi Pengujian

- Uji unit untuk fungsi murni pada game.js dan storage.js menggunakan Node.js dan assert.
- Uji fungsional manual untuk alur pengguna di peramban sesuai skenario Bab 2.
- Uji kasus tepi untuk nilai batas: waktu 0, input kosong, data rusak, nama panjang.
- Uji kompatibilitas pada Chrome, Edge, dan Firefox serta tampilan ponsel melalui DevTools.

7.2 Kasus Uji

Kode	Skenario	Masukan	Hasil yang diharapkan	Status
T-01	Hitung WPM normal	50 karakter benar, 30 detik	20 WPM	Lulus
T-02	Hitung WPM pada 60 detik	300 karakter benar, 60 detik	60 WPM	Lulus
T-03	WPM dengan waktu 0	50 karakter, 0 detik	0 (bukan Infinity)	Lulus
T-04	Akurasi normal	45 benar dari 50 diketik	90%	Lulus
T-05	Akurasi tanpa ketikan	0 benar dari 0 diketik	0% (bukan NaN)	Lulus
T-06	Penilaian kata salah	“datbase” vs “database”	3 karakter benar, 7 diketik, tidak benar	Lulus
T-07	Pengacakan tidak mengubah asli	Array [1,2,3,4,5]	Array asli tetap; hasil berisi elemen yang sama	Lulus
T-08	Timer mulai pada ketikan pertama	Buka halaman, belum mengetik	Timer belum berjalan	Lulus
T-09	Tombol Ulangi berulang kali	Tekan Ulangi 5 kali berturut-turut	Hanya satu timer berjalan	Lulus
T-10	Leaderboard kosong	`localStorage` belum ada data	Daftar kosong, tanpa error	Lulus
T-11	Data leaderboard rusak	Isi kunci dengan teks “{rusak”	Aplikasi tetap jalan, daftar kosong	Lulus
T-12	Batas 10 skor	Simpan 12 skor berbeda	Hanya 10 teratas tersimpan, terurut menurun	Lulus
T-13	Nama berisi tag HTML	<img src=x onerror=alert(1)>	Tampil sebagai teks, tidak dieksekusi	Lulus
T-14	Nama kosong	Nama dikosongkan lalu simpan	Tersimpan sebagai “Anonim”	Lulus

7.3 Contoh Uji Otomatis

Berkas berikut dijalankan dengan perintah node tests/game.test.js. Bila semua pernyataan lulus, program mencetak pesan keberhasilan; bila ada yang gagal, Node.js menampilkan nilai yang diharapkan dan nilai sebenarnya.

```
// tests/game.test.js  ->  jalankan: node tests/game.test.js
import assert from "node:assert/strict";
import { calculateWPM, calculateAccuracy, evaluateWord } from "../js/game.js";

assert.equal(calculateWPM(50, 30), 20);
assert.equal(calculateWPM(300, 60), 60);
assert.equal(calculateWPM(50, 0), 0);        // tidak boleh Infinity
assert.equal(calculateAccuracy(45, 50), 90);
assert.equal(calculateAccuracy(0, 0), 0);    // tidak boleh NaN
assert.deepEqual(evaluateWord("datbase", "database"),
                 { correct: 3, total: 7, isCorrect: false });

console.log("Semua pengujian lulus");
```



8. DEBUGGING: KASUS KESALAHAN DAN PERBAIKAN

Kesalahan berikut merupakan jenis kesalahan yang umum muncul pada kode hasil AI untuk aplikasi serupa. Setiap kasus telah dianalisis secara teknis pada saat penyusunan proposal (misalnya percobaan distribusi pengacakan pada Kasus 1 benar-benar dijalankan 200.000 kali), dan akan direproduksi serta dilengkapi tangkapan layar DevTools pada tahap implementasi. Tiap kasus disajikan dengan urutan: gejala, akar masalah, kode bermasalah, perbaikan, verifikasi, dan keputusan.

8.1 Kasus 1: Pengacakan Kata Bias

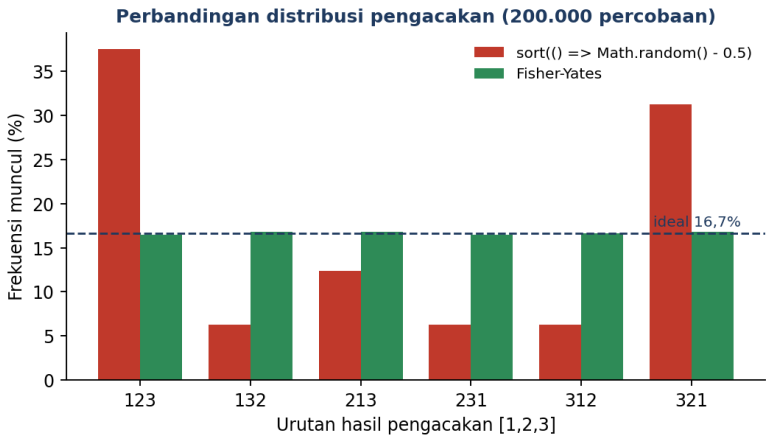
Gejala: kata-kata tertentu cenderung muncul di posisi yang sama, dan daftar kata asli ikut berubah.

Akar masalah: fungsi pembanding acak pada sort tidak konsisten sehingga hasilnya tidak seragam, dan sort mengubah array asli (in-place).

```
// Versi awal dari AI (bermasalah)
function shuffleWords(words) {
  return words.sort(() => Math.random() - 0.5);
}
```

Perbaikan: menggunakan algoritma Fisher-Yates pada salinan array (lihat shuffle pada Bab 6.2).

Verifikasi: array [1, 2, 3] diacak sebanyak 200.000 kali dengan kedua metode. Pada metode sort acak, urutan 123 muncul sekitar 37,5% (nilai ideal 16,7%), sedangkan pada Fisher-Yates seluruh enam urutan muncul sekitar 16,5% sampai 16,8%.



Keputusan: kode dari AI ditolak untuk bagian pengacakan dan diganti Fisher-Yates karena distribusi seragam adalah syarat keadilan permainan.

8.2 Kasus 2: WPM Menjadi NaN atau Infinity

Gejala: layar menampilkan “NaN WPM” atau “Infinity WPM” bila pengguna belum mengetik atau waktu 0.

Akar masalah: pembagian dengan nol pada rumus WPM yang tidak dijaga.

```
// Versi awal dari AI (bermasalah)
function calculateWPM(correctChars, elapsedSeconds) {
  return Math.round((correctChars / 5) / (elapsedSeconds / 60));
}
// calculateWPM(50, 0)  -> Infinity
// calculateWPM(0, 0)   -> NaN  (tampil "NaN WPM" di layar)
```

Perbaikan: menambahkan penjagaan if (elapsedSeconds <= 0) return 0 pada calculateWPM dan penjagaan serupa pada calculateAccuracy (lihat Bab 6.2).

Verifikasi: kasus uji T-03 dan T-05 harus menghasilkan 0 dan lulus pada tests/game.test.js.

Keputusan: fungsi dibuat selalu mengembalikan angka yang valid agar antarmuka tidak perlu memeriksa ulang.

8.3 Kasus 3: Timer Ganda Saat Tombol Ulangi Ditekan

Gejala: setelah menekan Ulangi beberapa kali, angka timer melompat-lompat dan layar hasil muncul sebelum waktunya.

Akar masalah: id dari setInterval tidak disimpan sehingga interval lama tidak pernah dihentikan. Beberapa interval berjalan bersamaan dan masing-masing memperbarui layar serta memanggil finishGame().

```
// Versi awal dari AI (bermasalah)
function startGame(duration) {
  let remaining = duration;
  setInterval(() => {          // id interval tidak disimpan
    remaining--;
    updateTimerUI(remaining);
    if (remaining <= 0) finishGame();
  }, 1000);
}
// Setiap klik "Ulangi" memanggil startGame() lagi,
// sehingga beberapa interval berjalan bersamaan.
```

Perbaikan: menyimpan id interval pada variabel, memanggil stopTimer() sebelum memulai timer baru, dan menghitung sisa waktu dari Date.now() (lihat Bab 6.3).

Verifikasi: tekan Ulangi lima kali berturut-turut (kasus uji T-09); angka timer harus turun satu per satu dan hasil hanya muncul sekali.

Keputusan: perbaikan dibuat minimal, hanya pada fungsi timer, tanpa menulis ulang berkas lain.

8.4 Kasus 4: Aplikasi Berhenti karena Data localStorage Rusak

Gejala: halaman tidak merespons dan konsol menampilkan SyntaxError atau TypeError saat membuka aplikasi.

Akar masalah: JSON.parse dipanggil tanpa penanganan kesalahan, dan nilai null tidak diperiksa.

```
// Versi awal dari AI (bermasalah)
function getScores() {
  return JSON.parse(localStorage.getItem("scores"));
}
// Jika data belum ada  -> JSON.parse(null) menghasilkan null, lalu scores.push() error
// Jika data rusak      -> SyntaxError dan seluruh halaman berhenti bekerja
```

Perbaikan: membungkus pembacaan dengan try/catch, memeriksa bahwa hasilnya berupa array, dan mengembalikan daftar kosong bila gagal (lihat Bab 6.4).

Verifikasi: kasus uji T-10 dan T-11 (data kosong dan data rusak) dijalankan melalui konsol peramban.

Keputusan: kegagalan membaca leaderboard tidak boleh mengganggu permainan, sehingga aplikasi memilih berjalan dengan daftar kosong.

8.5 Kasus 5: Celah XSS pada Nama Pemain

Gejala: nama pemain yang berisi tag HTML dieksekusi oleh peramban, misalnya memunculkan kotak peringatan.

Akar masalah: teks dari pengguna digabungkan ke string HTML dan dimasukkan melalui innerHTML.

```
// Versi awal dari AI (bermasalah)
function renderLeaderboard(scores) {
  const list = document.getElementById("leaderboard-list");
  list.innerHTML = "";
  scores.forEach((s, i) => {
    list.innerHTML += "<li>" + (i + 1) + ". " + s.name + " - " + s.wpm + " WPM</li>";
  });
}
// Nama pemain "<img src=x onerror=alert(1)>" akan dieksekusi sebagai HTML (XSS).
```

Perbaikan: membuat elemen dengan document.createElement dan mengisi teks melalui textContent (lihat Bab 6.5). Panjang nama juga dibatasi 20 karakter pada saveScore.

Verifikasi: kasus uji T-13; nama berisi tag HTML harus tampil apa adanya sebagai teks.

Keputusan: walaupun aplikasi hanya lokal, praktik aman tetap dibiasakan karena kode semacam ini sering disalin ke proyek lain.

8.6 Kasus 6: Galat CORS Saat Membuka index.html Langsung

Gejala: halaman kosong dan konsol menampilkan pesan galat CORS ketika berkas dibuka dengan klik ganda.

Akar masalah: type="module" mewajibkan berkas dimuat melalui protokol HTTP, bukan file://.

```
Access to script at 'file:///.../js/main.js' from origin 'null' has been
blocked by CORS policy: Cross origin requests are only supported for
protocol schemes: http, https, chrome, ...
```

Perbaikan: menjalankan aplikasi melalui server statis lokal (Live Server atau python -m http.server) dan mendokumentasikannya di README.

Verifikasi: halaman berjalan normal pada http://localhost:5500.

Keputusan: ES Modules dipertahankan demi struktur kode yang bersih, dengan konsekuensi langkah menjalankan yang ditulis jelas di README.

8.7 Ringkasan Kasus Debugging

No	Masalah	Akar masalah	Perbaikan	Bukti uji
1	Pengacakan bias	`sort` dengan pembanding acak	Fisher-Yates pada salinan array	Percobaan 200.000 kali; T-07
2	NaN / Infinity	Pembagian dengan nol	Penjagaan nilai tepi	T-03, T-05
3	Timer ganda	Id interval tidak disimpan	`stopTimer()` dan `Date.now()`	T-09
4	Crash data rusak	`JSON.parse` tanpa penanganan	`try/catch` dan validasi array	T-10, T-11
5	XSS nama pemain	`innerHTML`	`textContent`	T-13
6	Galat CORS	Modul dibuka lewat `file://`	Server statis lokal	Uji manual di peramban

9. REPOSITORI GITHUB

Seluruh hasil disimpan pada repositori GitHub dengan struktur yang rapi sehingga orang lain dapat menjalankan proyek hanya dengan mengikuti README. Nama repositori mengikuti format 11S24035-pabwe2026-typing-speed-battle.

9.1 Isi Repositori

```
typing-speed-battle/
├── index.html            # struktur halaman
├── css/
│   └── style.css         # tampilan & responsif
├── js/
│   ├── config.js         # pengaturan aplikasi
│   ├── words.js          # daftar kata (Umum & IT)
│   ├── game.js           # logika: acak, WPM, akurasi, timer
│   ├── storage.js        # leaderboard di localStorage
│   ├── ui.js             # render ke DOM
│   └── main.js           # alur & event listener
├── tests/
│   └── game.test.js      # uji fungsi murni (Node.js)
├── docs/
│   ├── proposal.docx     # dokumen proposal
│   └── images/           # tangkapan layar prompt & hasil
├── package.json          # hanya {"type": "module"}, tanpa dependensi
├── .gitignore
└── README.md
```

- **Kode program:** folder `js/`, `css/`, dan `index.html`.
- **Proposal dan bukti:** folder `docs/` berisi proposal dan tangkapan layar prompt serta hasil.
- **Pengujian:** folder `tests/` berisi uji otomatis fungsi murni.
- **Dokumentasi:** `README.md` berisi deskripsi, cara menjalankan, konfigurasi, dan catatan penggunaan AI.

9.2 Cara Menjalankan

```
# 1. Salin repositori
git clone https://github.com/<username>/<nama-repo>.git
cd <nama-repo>

# 2. Jalankan server statis lokal (pilih salah satu)
#    a) VS Code: pasang ekstensi "Live Server" -> klik "Go Live"
#    b) Python (tanpa instalasi tambahan):
python -m http.server 5500

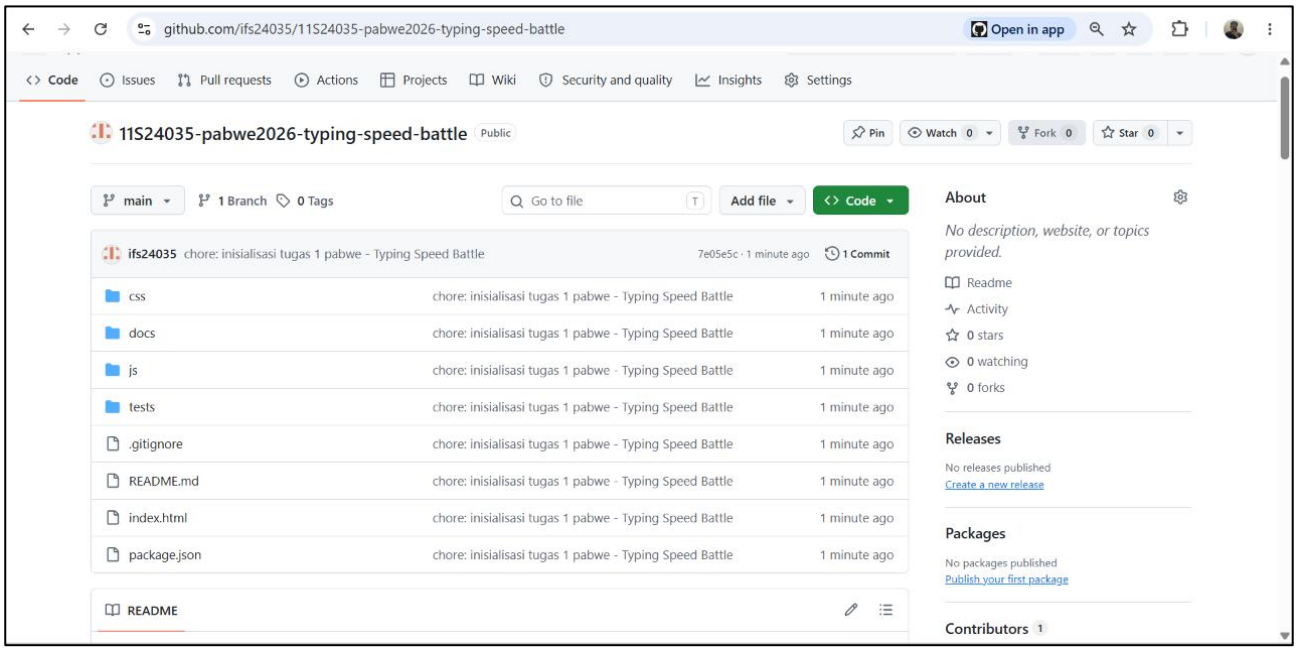
# 3. Buka di peramban
#    http://localhost:5500
```

9.3 Inisialisasi Repositori

```
git init
git add .
git commit -m "chore: inisialisasi proyek Typing Speed Battle"
git branch -M main
git remote add origin https://github.com/<username>/<nama-repo>.git
git push -u origin main
```

9.4 Konfigurasi

- Pengaturan aplikasi: seluruh nilai yang dapat diubah (durasi, jumlah kata, batas skor, panjang nama, kunci penyimpanan) berada di js/config.js (lihat Bab 6.1).
- Dependensi: tidak ada. Berkas package.json hanya berisi {"type": "module"} agar uji Node.js dapat memakai ES Modules.
- Publikasi (opsional): GitHub Pages dapat diaktifkan melalui Settings, Pages, pilih cabang main dan folder root.
- Peramban yang didukung: Chrome, Edge, dan Firefox versi terbaru.



KESIMPULAN

1. Aplikasi dibangun murni menggunakan HTML5, CSS3, dan ES6+ tanpa framework, serta menerapkan arsitektur ES Modules agar kode lebih terstruktur, modular, dan mudah dipelihara.
2. Pengacak Fisher-Yates memastikan hasil yang tidak bias, perhitungan WPM/akurasi aman dari galat division-by-zero, dan penanganan waktu berbasis real-time mencegah masalah timer drift maupun zombie interval.
3. Pengelolaan localStorage dilengkapi validasi defensif (try...catch) untuk mengantisipasi data rusak, serta penggunaan textContent alih-alih innerHTML untuk mencegah potensi serangan XSS.
4. Walau AI sangat membantu mempercepat pembuatan draf kode dan arsitektur, hasilnya tetap memerlukan evaluasi kritis serta uji verifikasi mandiri untuk menangani subtle bugs dan celah keamanan.
5. Seluruh berkas kode, konfigurasi, panduan server lokal, dan berkas pengujian otomatis telah tersusun rapi di repositori GitHub sehingga siap diuji dan direproduksi dengan transparan.

Refleksi Penggunaan AI

1. AI sangat membantu mempercepat perancangan dan penulisan draf kode, tetapi kualitas hasilnya sangat dipengaruhi kejelasan prompt.
2. Kode dari AI dapat tampak meyakinkan namun mengandung kesalahan halus (pengacakan bias, pembagian nol, celah keamanan) yang hanya terungkap melalui pengujian.
3. Meminta alasan dan cara verifikasi membuat AI menjadi alat belajar, bukan sekadar pembuat kode.
4. Tanggung jawab akhir tetap pada pengembang: kode hanya digunakan bila dipahami, diuji, dan dapat dijelaskan.

Pengembangan Lanjutan

1. Mode “Battle” dua pemain bergantian pada satu perangkat.
2. Grafik perkembangan WPM dari riwayat skor.
3. Tema gelap dan pilihan bahasa kata (Indonesia dan Inggris).
4. Pengunggahan leaderboard ke layanan daring pada tahap pengembangan berikutnya.

DAFTAR PUSTAKA

Anthropic. (2025). Prompt engineering overview. Anthropic Documentation. https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/overview

Fisher, R. A., & Yates, F. (1938). Statistical Tables for Biological, Agricultural and Medical Research. Oliver and Boyd.

GitHub. (n.d.). GitHub Pages documentation. https://docs.github.com/pages

Knuth, D. E. (1997). The Art of Computer Programming, Volume 2: Seminumerical Algorithms (3rd ed.). Addison-Wesley.

MDN Web Docs. (n.d.). JavaScript modules. Mozilla. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Modules

MDN Web Docs. (n.d.). Window: localStorage property. Mozilla. https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage

MDN Web Docs. (n.d.). Node: textContent property. Mozilla. https://developer.mozilla.org/en-US/docs/Web/API/Node/textContent

OWASP Foundation. (n.d.). Cross Site Scripting Prevention Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html